

8-Bit RISC Processor Design Using Verilog

VLSI Internship Project Report

Name: Vasamshetti Meghana

Intern ID: CITS691

Branch: ECE

Duration: 6 Weeks

Abstract

This project presents the design, implementation, simulation, and synthesis of an 8-Bit Reduced Instruction Set Computer (RISC) Processor using Verilog Hardware Description Language (HDL). The processor is designed based on a modular architecture comprising essential functional units such as the Program Counter (PC), Instruction Memory, Control Unit, Register File, Arithmetic Logic Unit (ALU), and Data Memory.

The proposed processor operates on 8-bit data and supports a fixed 16-bit instruction format to simplify instruction decoding and execution. The implemented instruction set includes arithmetic and logical operations such as ADD, SUB, AND, OR, and XOR. The Control Unit decodes instruction opcodes and generates the necessary control signals required for processor operation.

The processor follows a sequential execution flow consisting of instruction fetch, decode, execute, memory access, and write-back stages. The Register File stores operands and execution results, while the ALU performs arithmetic and logical computations based on the decoded instruction.

Functional verification of the design is carried out using a comprehensive Verilog test-bench in Vivado Design Suite. Simulation waveforms are analyzed to validate correct instruction execution, Program Counter operation, register updates, and ALU functionality. Furthermore, synthesis is performed to evaluate hardware resource utilization, implementation efficiency, and FPGA feasibility.

The obtained results demonstrate successful execution of all implemented instructions

with accurate computation and proper register updates. This project provides practical exposure to processor architecture, digital system design, Verilog HDL coding, simulation methodologies, synthesis techniques, and FPGA-oriented hardware implementation, thereby serving as a foundation for the development of advanced processor architectures and System-on-Chip (SoC) applications.

1 Introduction

The rapid advancement of embedded systems, Internet of Things (IoT) devices, consumer electronics, and System-on-Chip (SoC) technologies has increased the demand for efficient and high-performance processor architectures. Processors serve as the central computational units of digital systems and are responsible for executing instructions, controlling system operations, and managing data flow between hardware components.

Reduced Instruction Set Computer (RISC) architecture was developed to simplify processor design by utilizing a small set of simple and optimized instructions that can be executed efficiently. Unlike Complex Instruction Set Computer (CISC) architectures, which support a large number of complex instructions, RISC processors employ a limited instruction set, fixed instruction formats, and simplified control logic. These characteristics reduce hardware complexity, improve execution speed, and enable efficient pipelining.

An 8-bit RISC processor operates on 8-bit data and provides a compact and modular platform for understanding the fundamental concepts of processor design and computer architecture. Despite its simplicity, an 8-bit processor incorporates the essential building blocks found in modern processors, including instruction fetching, decoding, execution, memory access, and result storage.

The processor designed in this project consists of several functional modules, namely the Program Counter (PC), Instruction Memory, Control Unit, Register File, Arithmetic Logic Unit (ALU), and Data Memory. The Program Counter stores the address of the next instruction to be executed, while the Instruction Memory stores the program instructions. The Control Unit decodes the instruction opcode and generates the required control signals for processor operation.

The Register File contains general-purpose registers used for storing operands and computation results, whereas the ALU performs arithmetic and logical operations such as addition, subtraction, bitwise AND, OR, and XOR. Data Memory is included to support future implementation of memory access instructions such as LOAD and STORE.

The processor follows a sequential instruction execution cycle consisting of instruction fetch, decode, execute, memory access, and write-back stages. This modular design approach improves scalability, simplifies hardware implementation, and enables future enhancements such as pipelining, branching, interrupt handling, and support for additional instructions.

In this project, the 8-bit RISC processor is designed and implemented using Verilog

Hardware Description Language (HDL). Functional verification is performed using a comprehensive Verilog testbench in Vivado Design Suite. Simulation waveforms are analyzed to validate correct instruction execution and register updates, while synthesis is carried out to evaluate hardware resource utilization and implementation feasibility.

This project provides practical exposure to processor architecture, digital system design, hardware description languages, simulation methodologies, synthesis techniques, and FPGA-oriented implementation, thereby establishing a strong foundation for advanced processor and System-on-Chip (SoC) development.

2 Tools Used

The design, simulation, verification, and synthesis of the 8-Bit RISC Processor were carried out using industry-standard Electronic Design Automation (EDA) tools. These tools provide an efficient environment for hardware description, functional verification, RTL analysis, and implementation of digital systems.

- **Verilog Hardware Description Language (HDL)**

Verilog Hardware Description Language (HDL) is used to model and implement the complete architecture of the 8-Bit RISC Processor. It provides a hardware-level description of digital systems and enables designers to represent complex processor functionalities using behavioral, dataflow, and structural modeling techniques.

In this project, Verilog HDL is used to design and integrate various functional modules including the Program Counter, Instruction Memory, Control Unit, Register File, Arithmetic Logic Unit (ALU), and Data Memory. Verilog also enables the creation of testbenches for functional verification and supports simulation-driven design methodologies.

The use of Verilog HDL offers several advantages such as modularity, reusability, portability, and ease of debugging. Its compatibility with modern FPGA and ASIC design flows makes it one of the most widely adopted hardware description languages in digital system design.

- **Vivado Design Suite**

Vivado Design Suite is the primary development environment used for designing, simulating, synthesizing, and analyzing the 8-Bit RISC Processor. Developed by Xilinx, Vivado provides an integrated platform for FPGA-based digital design and verification.

The Verilog modules developed for the processor are imported into Vivado, where behavioral simulation is performed to verify correct instruction execution and overall processor functionality. The integrated waveform viewer is used to analyze

important signals such as the Program Counter, instruction bus, control signals, ALU outputs, and register values.

Vivado is also used to generate RTL schematics, which provide a graphical representation of the synthesized hardware architecture and internal module interconnections. The synthesis process converts the Verilog description into an optimized hardware netlist and provides detailed reports on resource utilization, timing performance, and implementation feasibility.

The comprehensive features offered by Vivado Design Suite simplify the hardware development process and enable efficient validation of processor architectures before deployment on FPGA platforms.

3 Objectives

The primary objectives of this project are:

- To understand the fundamentals of Reduced Instruction Set Computer (RISC) architecture.
- To study the instruction execution cycle including fetch, decode, execute, memory access, and write-back stages.
- To design an 8-bit processor using modular hardware blocks.
- To implement arithmetic and logical instructions using an Arithmetic Logic Unit (ALU).
- To develop a register file for storing operands and computation results.
- To design a control unit capable of generating control signals based on instruction opcodes.
- To verify processor functionality using simulation waveforms.
- To perform synthesis and analyze FPGA resource utilization.
- To gain practical experience in processor design using Verilog HDL and Vivado Design Suite.

4 8-Bit RISC Processor Overview

A Reduced Instruction Set Computer (RISC) processor is a type of microprocessor architecture designed to execute a small and optimized set of instructions efficiently. Unlike Complex Instruction Set Computer (CISC) architectures, RISC processors focus on

simple instructions with fixed execution cycles, resulting in improved performance and simplified hardware implementation.

The proposed 8-bit RISC processor operates on 8-bit data and executes instructions stored in instruction memory. Each instruction follows a fixed 16-bit format consisting of opcode fields, source register addresses, and destination register addresses.

The processor architecture includes a Program Counter (PC), Instruction Memory, Control Unit, Register File, Arithmetic Logic Unit (ALU), and Data Memory. These modules work together to fetch instructions, decode operation types, perform computations, and store results.

The processor supports arithmetic and logical instructions such as ADD, SUB, AND, OR, and XOR. The modular design approach improves scalability and enables future expansion with additional instructions such as MOVI, LOAD, STORE, branch, and jump operations.

Due to its simplicity and hardware efficiency, the 8-bit RISC processor is suitable for educational purposes, embedded systems, FPGA prototyping, and low-power computing applications.

5 Processor Architecture

The architecture of the 8-bit RISC processor is designed using multiple interconnected modules that collectively perform instruction execution.

The Program Counter (PC) stores the address of the next instruction to be executed and increments sequentially after each instruction cycle.

Instruction Memory stores the program instructions and provides the current instruction based on the Program Counter value.

The Control Unit decodes the instruction opcode and generates the required control signals for selecting ALU operations, enabling register writes, and controlling memory access.

The Register File consists of eight general-purpose 8-bit registers used to store input operands and intermediate results.

The Arithmetic Logic Unit (ALU) performs arithmetic and logical operations such as addition, subtraction, bitwise AND, OR, and XOR.

Data Memory is used to store intermediate values and supports future implementation of LOAD and STORE instructions.

The processor follows a sequential execution flow consisting of instruction fetch, decode, execute, memory access, and write-back stages.

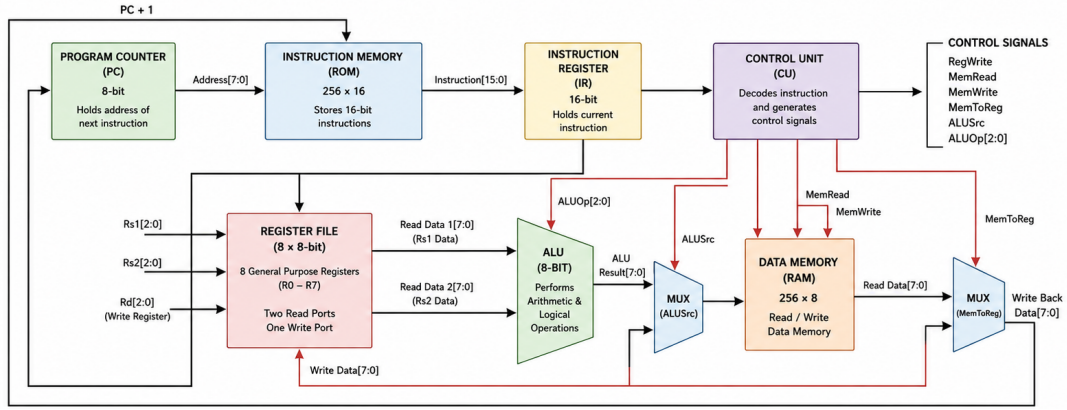


Figure 1: Block Diagram of the 8-Bit RISC Processor

6 Functional Blocks

The major functional blocks of the processor are described below.

6.1 Program Counter

The Program Counter stores the address of the current instruction and increments after every clock cycle to fetch the next instruction.

6.2 Instruction Memory

Instruction Memory stores the processor instructions in a predefined format and provides the instruction corresponding to the Program Counter value.

6.3 Control Unit

The Control Unit decodes the opcode field and generates control signals required for processor operation.

6.4 Register File

The Register File consists of eight 8-bit general-purpose registers used to store operands and execution results.

6.5 Arithmetic Logic Unit

The ALU performs arithmetic and logical operations based on the control signals generated by the Control Unit.

6.6 Data Memory

The Data Memory stores temporary data values and supports memory access operations.

7 Instruction Format

Each instruction in the processor is 16 bits wide and consists of multiple fields as shown below.

Bits [15:12]	Bits [11:9]	Bits [8:6]	Bits [5:3]
Opcode	Destination Register	Source Register 1	Source Register 2

Table 1: 16-Bit Instruction Format

The opcode field specifies the operation to be performed, while the remaining fields identify the source and destination registers.

8 Instruction Set Architecture

Opcode	Instruction	Operation
0000	ADD	$Rd = Rs1 + Rs2$
0001	SUB	$Rd = Rs1 - Rs2$
0010	AND	$Rd = Rs1 \& Rs2$
0011	OR	$Rd = Rs1 \text{ --- } Rs2$
0100	XOR	$Rd = Rs1 \oplus Rs2$
0101	LOAD	$Rd = \text{MEM}[\text{Address}]$
0110	STORE	$\text{MEM}[\text{Address}] = Rs$
0111	MOVI	$Rd = \text{Immediate}$

Table 2: Instruction Set of the 8-Bit RISC Processor

9 FSM Design

A Finite State Machine (FSM) is used to control the sequential operation of the 8-Bit RISC Processor. The FSM coordinates the execution of instructions by generating appropriate control signals and managing the flow of data between the processor modules.

The processor follows a multi-cycle execution approach in which each instruction passes through a series of states. These states ensure that instructions are fetched, decoded, executed, and stored correctly while maintaining synchronization among the functional units.

The FSM consists of the following five states:

- **FETCH** – In this state, the Program Counter (PC) provides the address of the next instruction, which is fetched from the Instruction Memory.
- **DECODE** – The fetched instruction is decoded by the Control Unit. The opcode and register addresses are extracted, and the required control signals are generated.
- **EXECUTE** – The Arithmetic Logic Unit (ALU) performs the specified arithmetic or logical operation using the operands obtained from the Register File.
- **MEMORY** – This state handles data memory access operations such as LOAD and STORE instructions. For the currently implemented instruction set, this state is reserved for future enhancements.
- **WRITEBACK** – The result generated by the ALU or Data Memory is written back into the destination register. The Program Counter is then updated to point to the next instruction.

The FSM ensures proper sequencing of processor operations, minimizes hardware complexity, and enables future expansion of the instruction set architecture.

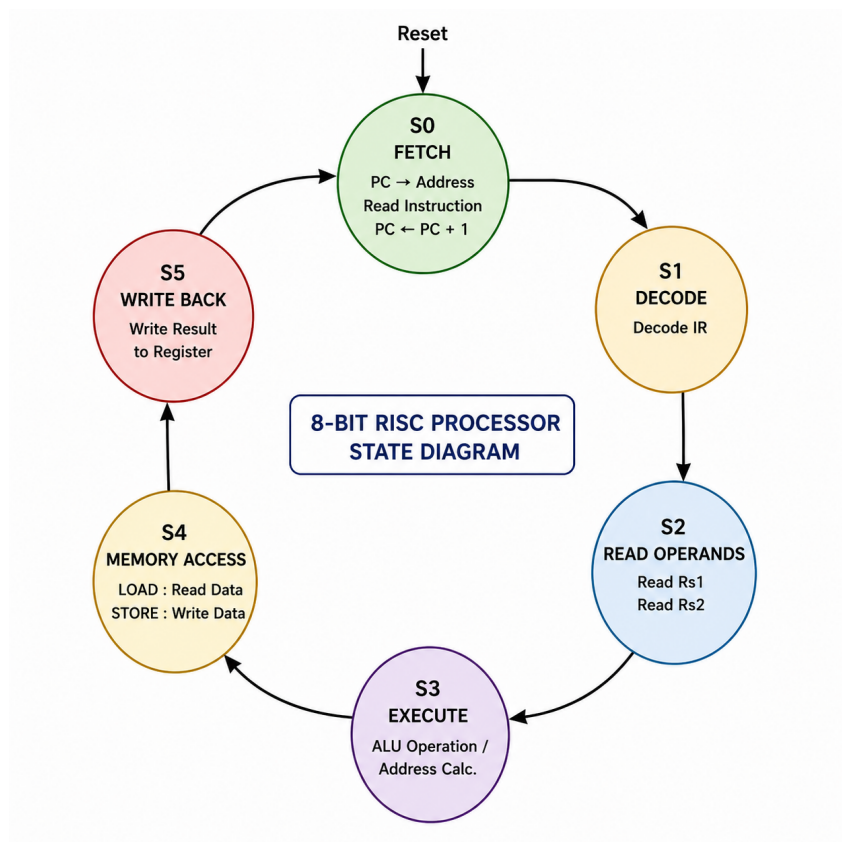


Figure 2: FSM State Diagram of the 8-Bit RISC Processor

10 Verilog HDL Implementation

10.1 Program Counter

```
module program_counter(  
    input clk,  
    input reset,  
    output reg [7:0] pc  
);  
  
always @(posedge clk or posedge reset) begin  
    if (reset)  
        pc <= 8'd0;  
    else  
        pc <= pc + 1'b1;  
    end  
  
endmodule
```

10.2 Instruction Memory

```
module instruction_memory(  
    input [7:0] addr,  
    output [15:0] instruction  
);  
  
reg [15:0] mem [0:255];  
integer i;  
initial begin  
    for (i = 0; i < 256; i = i + 1)  
        mem[i] = 16'd0;  
  
    // ADD R3 = R1 + R2  
    mem[0] = 16'b0000_011_001_010_000;  
  
    // SUB R4 = R1 - R2  
    mem[1] = 16'b0001_100_001_010_000;  
  
    // AND R5 = R1 & R2  
    mem[2] = 16'b0010_101_001_010_000;  
  
    // OR R6 = R1 | R2
```

```

    mem[3] = 16'b0011_110_001_010_000;

    // XOR R7 = R1 ^ R2
    mem[4] = 16'b0100_111_001_010_000;
end
assign instruction = mem[addr];

endmodule

```

10.3 control unit

```

module control_unit(
    input [3:0] opcode,

    output reg reg_write,
    output reg mem_read,
    output reg mem_write,
    output reg [2:0] alu_op
);

always @(*) begin

    reg_write = 0;
    mem_read  = 0;
    mem_write = 0;
    alu_op    = 3'b000;

    case(opcode)

        4'b0000: begin
            reg_write = 1;
            alu_op    = 3'b000;
        end

        4'b0001: begin
            reg_write = 1;
            alu_op    = 3'b001;
        end

        4'b0010: begin
            reg_write = 1;
            alu_op    = 3'b010;

```

```

        end

        4'b0011: begin
            reg_write = 1;
            alu_op = 3'b011;
        end

        4'b0100: begin
            reg_write = 1;
            alu_op = 3'b100;
        end

        4'b0101: begin
            reg_write = 1;
            mem_read = 1;
        end

        4'b0110: begin
            mem_write = 1;
        end

        4'b0111: begin
            reg_write = 1;
        end

    endcase

end

endmodule

```

10.4 Register File

```

module register_file(
    input clk,
    input we,
    input [2:0] rs1,
    input [2:0] rs2,
    input [2:0] rd,
    input [7:0] write_data,

    output [7:0] read_data1,

```

```

        output [7:0] read_data2
    );

    reg [7:0] regs [0:7];
    integer i;

    // Initialize registers
    initial begin
        for (i = 0; i < 8; i = i + 1)
            regs[i] = 8'd0;

        // Temporary values for testing
        regs[1] = 8'd10;
        regs[2] = 8'd5;
    end

    assign read_data1 = regs[rs1];
    assign read_data2 = regs[rs2];

    always @(posedge clk) begin
        if (we)
            regs[rd] <= write_data;
    end

endmodule

```

10.5 ALU

```

module alu(
    input [7:0] a,
    input [7:0] b,
    input [2:0] alu_op,

    output reg [7:0] result
);

always @(*) begin
    case (alu_op)
        3'b000: result = a + b;
        3'b001: result = a - b;
        3'b010: result = a & b;
        3'b011: result = a | b;
    end
end

```

```

        3'b100: result = a ^ b;
        default: result = 8'd0;
    endcase
end

endmodule

```

10.6 data memory

```

module data_memory(
    input clk,
    input mem_write,
    input mem_read,
    input [7:0] addr,
    input [7:0] write_data,

    output reg [7:0] read_data
);

reg [7:0] mem [0:255];

always @(posedge clk) begin
    if (mem_write)
        mem[addr] <= write_data;
end

always @(*) begin
    if (mem_read)
        read_data = mem[addr];
    else
        read_data = 8'd0;
end

endmodule

```

10.7 top module

```

module risc8_top(
    input clk,
    input reset,

```

```

        output [7:0] pc_out,
        output [15:0] instr_out,
        output [7:0] alu_out,
        output [7:0] reg_data1_out,
        output [7:0] reg_data2_out
    );

    wire [7:0] pc;
    wire [15:0] instr;

    wire [3:0] opcode;
    wire [2:0] rd, rs1, rs2;

    wire reg_write;
    wire mem_read;
    wire mem_write;

    wire [2:0] alu_op;

    wire [7:0] reg_data1;
    wire [7:0] reg_data2;

    wire [7:0] alu_result;
    wire [7:0] mem_data;

    // ----- INSTRUCTION DECODE -----
    assign opcode = instr[15:12];
    assign rd     = instr[11:9];
    assign rs1    = instr[8:6];
    assign rs2    = instr[5:3];

    // ----- OUTPUT ASSIGNMENTS -----
    assign pc_out      = pc;
    assign instr_out   = instr;
    assign alu_out     = alu_result;
    assign reg_data1_out = reg_data1;
    assign reg_data2_out = reg_data2;

    // ----- PROGRAM COUNTER -----

```

```

program_counter PC (
    .clk(clk),
    .reset(reset),
    .pc(pc)
);

// ----- INSTRUCTION MEMORY -----
instruction_memory IM (
    .addr(pc),
    .instruction(instr)
);

// ----- CONTROL UNIT -----
control_unit CU (
    .opcode(opcode),
    .reg_write(reg_write),
    .mem_read(mem_read),
    .mem_write(mem_write),
    .alu_op(alu_op)
);

// ----- REGISTER FILE -----
register_file RF (
    .clk(clk),
    .we(reg_write),
    .rs1(rs1),
    .rs2(rs2),
    .rd(rd),
    .write_data(mem_read ? mem_data : alu_result),
    .read_data1(reg_data1),
    .read_data2(reg_data2)
);

// ----- ALU -----
alu ALU (
    .a(reg_data1),
    .b(reg_data2),
    .alu_op(alu_op),
    .result(alu_result)
);

```

```

);

// ----- DATA MEMORY -----
data_memory DM (
    .clk(clk),
    .mem_write(mem_write),
    .mem_read(mem_read),
    .addr(alu_result),
    .write_data(reg_data2),
    .read_data(mem_data)
);

endmodule

```

11 Testbench Design

```

`timescale 1ns/1ps

module tb_risc8_top;

reg clk;
reg reset;

risc8_top DUT(
    .clk(clk),
    .reset(reset)
);

always #5 clk = ~clk;

initial begin
    clk = 0;
    reset = 1;

    #20;
    reset = 0;

    #200;
    $finish;
end

```



```
end  
  
endmodule
```

12 Waveform Analysis

Simulation waveforms are used to verify the correct functionality of the 8-Bit RISC Processor designed using Verilog HDL. The waveforms confirm proper instruction fetching, decoding, execution, and register updates.

Initially, registers R1 and R2 are initialized with decimal values 10 and 5, respectively. The processor fetches instructions sequentially from Instruction Memory based on the Program Counter value.

The Control Unit decodes each instruction and generates the required ALU control signals. The ALU performs arithmetic and logical operations, and the computed result is written back into the destination register.

The simulation confirms correct Program Counter incrementation, instruction decoding, ALU operation selection, and register updates.

The final register values obtained after simulation are as follows:

- R1 = 10
- R2 = 5
- R3 = 15 (ADD)
- R4 = 5 (SUB)
- R5 = 0 (AND)
- R6 = 15 (OR)
- R7 = 15 (XOR)

The waveform analysis confirms successful execution of all instructions and validates the correctness of the processor design.

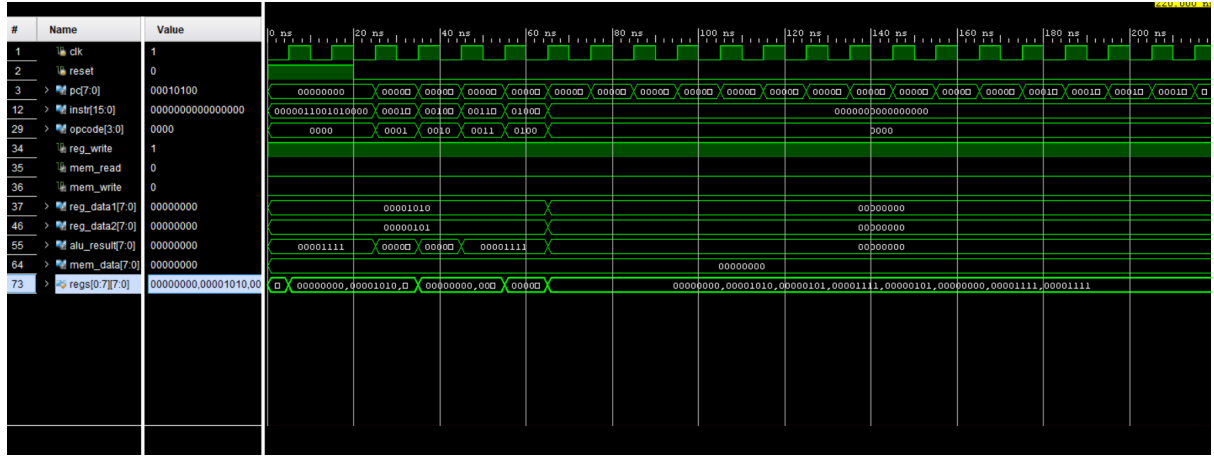


Figure 3: Simulation Waveform of the 8-Bit RISC Processor

13 RTL Schematic

The Register Transfer Level (RTL) schematic represents the synthesized hardware structure of the 8-Bit RISC Processor generated by Vivado Design Suite. The RTL view provides a graphical representation of the internal architecture and illustrates the inter-connection between the major functional modules of the processor.

The schematic includes essential components such as the Program Counter (PC), Instruction Memory, Control Unit, Register File, Arithmetic Logic Unit (ALU), and Data Memory. It also shows the datapath and control signals responsible for instruction execution and data transfer between the modules.

The Program Counter supplies instruction addresses to the Instruction Memory, while the Control Unit decodes instruction opcodes and generates the required control signals. The Register File provides input operands to the ALU, which performs arithmetic and logical operations based on the selected instruction. The execution results are then written back to the destination registers through the write-back path.

The RTL schematic is generated after the synthesis process and helps verify that the Verilog HDL description has been correctly translated into hardware components. It also assists in understanding the processor datapath, identifying module connectivity, and analyzing the overall hardware implementation.

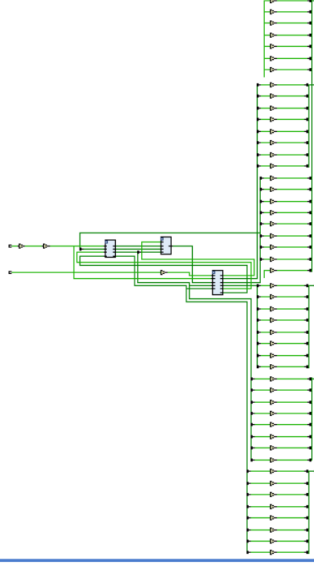


Figure 5: Synthesis Result of the 8-Bit RISC Processor in Vivado Design Suite

Parameter	Description / Observation
Flip-Flops	Used for storing state information, register contents, and Program Counter values.
Slices	Represent grouped FPGA resources utilized by the processor design.
Input/Output Ports	Used for clock, reset, and external communication signals.
Power Consumption	Low power utilization due to the compact architecture and simple instruction set.

Table 3: Synthesis Report Summary of the 8-Bit RISC Processor

15 Results

The 8-Bit RISC Processor was successfully designed, implemented, and verified using Verilog HDL in Vivado Design Suite. Functional verification was performed through behavioral simulation to validate the correct execution of processor instructions and overall system operation.

The processor successfully executed a sequence of arithmetic and logical instructions, including ADD, SUB, AND, OR, and XOR. The simulation results confirmed proper instruction fetching from Instruction Memory, accurate instruction decoding by the Control Unit, and correct operation of the Arithmetic Logic Unit (ALU).

The Program Counter incremented sequentially after each instruction cycle, enabling continuous instruction execution. The Register File correctly supplied input operands to

the ALU and stored the computed results in the designated destination registers during the write-back stage.

The initial values of registers R1 and R2 were set to 10 and 5, respectively. Based on these input values, the processor generated the following outputs:

- $R3 = 15$ for the ADD operation
- $R4 = 5$ for the SUB operation
- $R5 = 0$ for the AND operation
- $R6 = 15$ for the OR operation
- $R7 = 15$ for the XOR operation

The obtained results matched the expected outputs for all implemented instructions, confirming the correctness of the processor datapath and control logic.

Overall, the simulation and synthesis results demonstrate that the proposed 8-Bit RISC Processor operates reliably and efficiently, making it suitable for educational applications, FPGA prototyping, and future extension toward more advanced processor architectures.

16 Conclusion

The 8-bit RISC processor was successfully designed, implemented, and verified using Verilog HDL and Vivado Design Suite.

The processor correctly executed arithmetic and logical instructions including ADD, SUB, AND, OR, and XOR. Simulation results confirmed proper instruction fetching, decoding, ALU operation, and register updates.

The modular architecture consisting of the Program Counter, Instruction Memory, Control Unit, Register File, and ALU enabled efficient hardware implementation and simplified verification.

Synthesis results demonstrated that the processor utilizes minimal FPGA resources while maintaining reliable operation.

Overall, this project provides valuable practical experience in processor architecture, digital system design, hardware description languages, simulation methodologies, and FPGA-based implementation techniques.

17 Advantages

- Simple and modular architecture
- Fast instruction execution

- Efficient hardware utilization
- Easy FPGA implementation
- Scalable design for future enhancements

18 Applications

- Embedded Systems
- FPGA-Based Computing
- Educational Processor Design
- IoT Devices
- SoC Prototyping

19 Future Scope

- Implementation of MOVI, LOAD, and STORE instructions
- Support for branch and jump instructions
- Pipeline architecture implementation
- Interrupt handling mechanism
- FPGA deployment and hardware validation
- Extension to 16-bit and 32-bit architectures

20 References

- Computer Organization and Design by Patterson and Hennessy
- Verilog HDL by Samir Palnitkar
- Digital Design by Morris Mano
- Vivado Design Suite User Guide